RELATÓRIO TÉCNICO DE IMPLEMENTAÇÃO / VERSÃO 0.3

MicroLearn

Sprint 3 — Avaliações, notas e recomendações

Objetivo do incremento

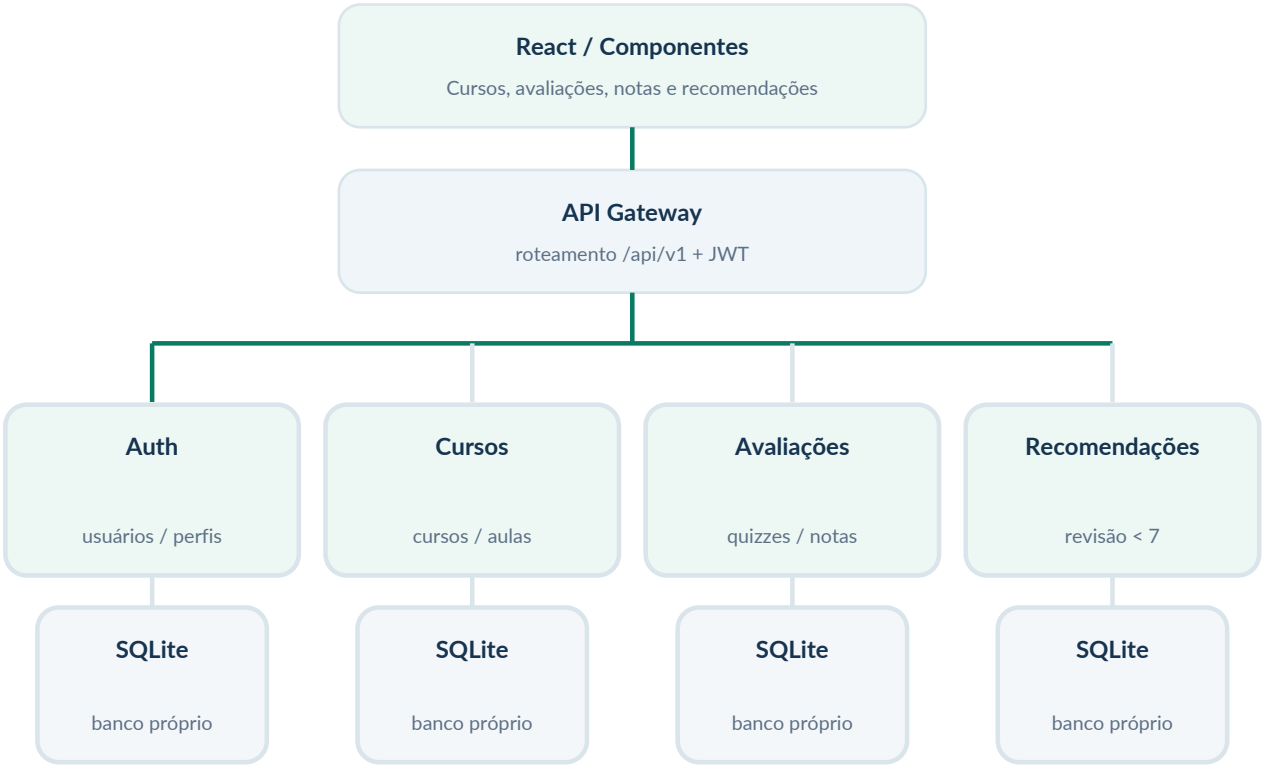
Implementar a avaliação objetiva e o acompanhamento de aprendizagem no Ambiente Virtual de Aprendizagem. O professor cria questionários vinculados a uma aula; o aluno matriculado responde, recebe uma nota e consulta recomendações de revisão quando o desempenho é inferior a 7,0. O incremento preserva autenticação, cursos e matrículas das sprints anteriores.

Entrega	Situação verificada
Criação de questionários e questões	Implementada e validada com testes
Submissão, correção, nota 0 a 10	Implementadas, com uma tentativa por aluno
Consulta individual e notas do professor	Implementada, com autorização por papel e vínculo
Recomendações de revisão	Implementadas com regra objetiva (nota inferior a 7,0)
React: questionários, notas e recomendações	Código implementado; análise sintática aprovada
Docker Compose	Configuração ampliada; execução no Docker pendente

Validações observadas: 21 testes Python aprovados; smoke HTTP passando por Gateway e quatro microsserviços; 23 arquivos JS/JSX sem erros sintáticos. O frontend ainda exige build npm e navegação real em navegador para validação final.

1. Arquitetura e isolamento dos serviços

Autenticação, Cursos, Avaliações e Recomendações continuam executados de forma independente, com a API Gateway como ponto único de acesso do frontend. Avaliações e Recomendações possuem seus próprios bancos SQLite, sem consulta direta a tabelas de outros domínios.



Chamadas internas HTTP: Avaliações → Auth/Cursos; Recomendações → Auth/Avaliações/Cursos

As relações entre domínios utilizam identificadores UUID e consultas HTTP autenticadas: Avaliações valida a conta ativa no Auth e o acesso à aula no Cursos; Recomendações consulta as notas no Avaliações e os materiais disponíveis no Cursos. O resultado é recalculado por solicitação e armazenado em cache específico do serviço.

1.1. Estruturas de persistência

Microsserviço / tabela	Campos e restrições relevantes
Avaliações / quizzes	ID, course_id, teacher_id, lesson_id, título e data
Avaliações / questions	ID, quiz_id, posição, texto, alternativas JSON e índice correto
Avaliações / attempts	ID, quiz_id, student_id, respostas JSON, acertos, total, nota; UNIQUE(quiz_id,student_id)
Recomendações / recommendations	ID, student_id, attempt_id, curso, aula, nota, título; UNIQUE(student_id,attempt_id)

2. API REST e regras de negócio

Todos os endpoints são oferecidos pelo prefixo /api/v1 na Gateway. Os exemplos de URL a seguir são relativos a esse prefixo.

Método / endpoint	Responsabilidade
POST /assessments/courses/{id}/quizzes	Professor autor cria avaliação com questões e aula associada
GET /assessments/courses/{id}/quizzes	Autor ou aluno matriculado lista avaliações do curso
GET /assessments/quizzes/{id}	Exibe questões sem revelar índices corretos
POST /assessments/quizzes/{id}/submit	Aluno matriculado envia respostas e recebe nota
GET /assessments/results/mine	Aluno consulta apenas suas notas
GET /assessments/courses/{id}/results	Professor autor consulta resultados do curso
DELETE /assessments/quizzes/{id}	Autor exclui avaliação ainda não respondida
GET /recommendations/mine	Aluno consulta aulas sugeridas para revisão

2.1. Regras implementadas

- O professor associa cada questionário a uma aula existente no próprio curso e escolhe a resposta correta de cada questão.
- Cada avaliação contém de 1 a 20 questões, com 2 a 5 alternativas, e utiliza somente respostas objetivas.
- A API que fornece as questões não retorna o campo correct_index, evitando revelar o gabarito antes da submissão.
- Somente alunos matriculados podem responder. O banco restringe uma tentativa por quiz_id + student_id; duplicidades retornam HTTP 409.
- Cada questão tem o mesmo peso: nota = arredondar(10 × acertos / questões, 2). Notas inferiores a 7,0 disparam recomendações de revisão.
- O professor vê resultados apenas dos seus cursos; um aluno só consulta resultados de sua própria conta.

3. Interface React baseada em componentes

Foram integradas novas páginas ao roteador existente, mantendo AuthContext e o cliente HTTP centralizado da Sprint 1. O componente AssessmentEditor permite cadastrar questões, alternativas, resposta correta e aula de revisão.

Página ou componente	Utilização
/cursos/:id/avaliacoes	Professor: criação e notas; aluno: lista de questionários
/avaliacoes/:quizId	Alunos respondem questões e recebem nota; autor visualiza prévia
/desempenho	Aluno consulta notas e recomendações
AssessmentEditor	Formulário reutilizável de questionários e questões
assessmentsApi / recommendationsApi	Cliente HTTP compartilhado com a Gateway
CourseDetail / Navbar / Dashboard	Navegação integrada aos módulos da Sprint 3

3.1. Fluxo demonstrável

Professor autentica-se → acessa um curso próprio → cria a avaliação e indica a aula correspondente → aluno matriculado abre o questionário → responde as questões → recebe nota 5,00 / 10,00 → abre Desempenho → visualiza recomendação da aula vinculada. Os resultados do professor ficam disponíveis no gerenciamento de avaliações.

3.2. Exemplo de correção

Questão	Resposta correta	Resposta do aluno	Resultado
2 + 2	Alternativa 2	Alternativa 2	Acerto
3 + 3	Alternativa 2	Alternativa 1	Erro
Nota final	2 questões	1 acerto	5,00 / 10

O sistema de recomendações aplica a regra nota < 7,0 e indica a aula configurada pelo professor. Não utiliza IA generativa; é um recomendador transparente baseado em regras.

4. Testes e evidências de execução

Foram utilizados bancos temporários e processos HTTP reais para evitar interferência de testes no banco de desenvolvimento.

Verificação	Resultado observado
pytest -q	21 testes aprovados: Auth, Gateway, Cursos, Avaliações e Recomendações
smoke_processes.py	Gateway e quatro microsserviços ativos por HTTP em processos separados
Criação de questionário pelo professor	HTTP 201; aula vinculada validada por API
Consulta do aluno ao questionário	HTTP 200; resposta sem campo de gabarito
Resposta com 1 acerto em 2 questões	HTTP 201; nota 5,00 registrada
Reenvio do mesmo questionário	HTTP 409; duplicidade impedida
Recomendação para nota 5,00	Aula correta retornada para revisão
Aluno sem matrícula	HTTP 403 ao consultar questionário alheio
Sintaxe JS/JSX	23 arquivos analisados; zero erros reportados
npm run build e execução Docker	Ainda não executados neste ambiente

4.1. Arquivos de evidência

No diretório docs/ do ZIP estão os arquivos pytest_results_sprint3.txt, smoke_test_results_sprint3.txt e VALIDACAO_SPRINT3.md, além dos relatórios anteriores.

4.2. Limitações técnicas conhecidas

- A exclusão de aulas ou cursos na Sprint 2 não executa limpeza transacional de questionários e notas em outros bancos. É necessário definir política de histórico e eventos distribuídos.
- Um questionário respondido não pode ser excluído; questionários ainda não permitem edição de conteúdo depois de criados.
- O cache de recomendações é recalculado no acesso; a indisponibilidade de dependências HTTP impede novas recomendações até restabelecer a comunicação.
- Os testes Python não substituem a verificação de interface no navegador, o build Vite, o teste visual responsivo e a validação Docker no computador do grupo.

5. Execução e próxima etapa

A Sprint 3 é um incremento sobre as versões anteriores e não requer reinicialização manual de bancos. As novas tabelas são criadas na inicialização dos serviços com CREATE TABLE IF NOT EXISTS.

Sequência	Comando / ação sugerida no Windows
1	Extrair MicroLearn_Sprint3_Codigo_Funcional.zip e abrir a pasta microlearn
2	Copy-Item .env.example .env
3	Configurar JWT_SECRET forte e, para demonstração, ADMIN_EMAIL e ADMIN_PASSWORD
4	docker compose up --build -d
5	docker compose ps; abrir http://localhost:5173
6	Cadastrar aluno, promover professor pelo administrador; criar curso e aula
7	Criar avaliação, matricular aluno, responder e consultar Desempenho
8	Opcional: python -m pytest -q; python tests/smoke_processes.py

5.1. Entrega da Sprint 3

O ZIP inclui o código atualizado, componentes React, configurações do Compose, testes, evidências de validação e este relatório. A API Gateway continua disponível em http://localhost:8000/docs; a lista de serviços pode ser consultada em /api/v1/status.

5.2. Próxima etapa: Sprint 4

Finalizar testes end-to-end no navegador e com Docker, revisar fluxos e segurança, preparar capturas das telas e dos microsserviços em execução, atualizar o README, documentar cada componente e organizar o relatório final da disciplina para publicação no GitHub.

Referências e rastreabilidade

Este relatório foi elaborado a partir do código-fonte da Sprint 3, dos requisitos e backlogs do MicroLearn 2026.2 e dos documentos anteriores de modelagem. Referência técnica principal: FastAPI, documentação OpenAPI, React, SQLite e Docker Compose, conforme ferramentas utilizadas no projeto.